

LLMsVerifier

LLMsVerifier

.تأیید. نظارت. بهینه‌سازی

خلاصه

یک پلتفرم سازمانی برای تأیید، نظارت و بهینه‌سازی مدل‌های زبانی بزرگ در میان ارائه‌دهندگان مختلف است که بر اساس آزمون اجباری «آیا کد من را می‌بینی؟» ساخته شده است تا تنها مدل‌هایی که ثابت شده واقعاً کار می‌کنند، به‌عنوان قابل استفاده علامت‌گذاری یا صادر شوند.

توضیح کوتاه

که مدل‌های زبانی بزرگ را در میان چندین ارائه‌دهنده تأیید، معیارسنجی، نظارت و Go پلتفرم بهینه‌سازی می‌کند. هر مدل پیش از استفاده باید از آزمون اجباری دیدن کد عبور کند؛ سپس آزمایش‌های تأخیر، استریم، فراخوانی تابع، بینایی و تعبیه را پشت سر می‌گذارد و تنها تنظیمات تأییدشده را برای صادر می‌کند CLI و AI ابزارهای

توضیح مفصل

در میان LLM یک پلتفرم جامع برای تأیید، نظارت و بهینه‌سازی عملکرد LLMsVerifier است و در این زمینه هیچ سازشی نمی‌پذیرد تأیید اجباری ارائه‌دهندگان مختلف است. اصل محوری آن پیش از آنکه هر مدلی به‌عنوان قابل استفاده علامت‌گذاری شود — یا اجازه ورود به تنظیمات صادرشده را پیدا کند — باید آزمون «آیا کد من را می‌بینی؟» را با موفقیت پشت سر بگذارد؛ آزمونی که با ارسال به ارائه‌دهنده و تحلیل پاسخ‌ها برای درک واقعی (و نه صرفاً بازتابی شبیه HTTP درخواست‌های واقعی به واقعیت) انجام می‌شود. مدلی که نتواند به‌طور قطعی ورودی شمارا ببیند و درک کند، هرگز برچسب قابل استفاده را دریافت نخواهد کرد. پس از عبور از این مرحله، موتور تأییدکننده مجموعه‌ای کامل از «آزمایش‌های قابلیت اجرا می‌کند — وجود، پاسخگویی، تأخیر، استریم، فراخوانی تابع، بینایی و تبدیل می‌کند JSON و موتور گزارش‌دهنده نتایج را به گزارش‌های مارک‌داون و — embeddings که بر اساس آن‌ها می‌توان اقدام کرد.

را بر REST API رابط کاربری متنی، وب و CLI این سیستم مدولار و رویدادمحور است و رابط‌های پایه هسته‌ای متشکل از موتور تأییدکننده، موتور گزارش‌دهنده و مدیر تنظیمات ارائه می‌دهد. اما کار این سیستم به تأیید محدود نمی‌شود. لایه‌های پیشرفته‌تر الگوی سرپرست/کارگرا برای تجزیه وظایف مبتنی را برای جلوگیری از افت LLM اضافه می‌کنند، مدیریت زمینه با پنجره لغزان و خلاصه‌سازی LLM بر کیفیت جلسات طولانی، نقطه‌گذاری ابری، و سیستمی برای بازیابی پس از شکست با مدارشکن‌ها و

مسیریابی مبتنی بر تأخیر فراهم می‌آورند. زیرساخت پیرامونی آن برای محیط‌های عملیاتی طراحی شده است: اتوبوس رویداد مبتنی بر انتشار/اشتراک، زمان‌بندی کرون، تشخیص قیمت/محدودیت‌ها، پایگاه را به (llmsvd) و سیستم صافرات. یک قرارداد نام‌گذاری مشخصه، پسوند RAG برای vector داده هر ارائه‌دهنده/مدل تولیدشده اضافه می‌کند تا خروجی تأییدشده در نگاه اول قابل ردیابی باشد و هرگز با خروجی‌های تأییدنشده اشتباه گرفته نشود — و تنها مدل‌های تأییدشده در تنظیمات صادرشده برای نوشته می‌شوند. این سیستم Claude Code و Crush و OpenCode مانند AI CLI ابزارهای Docker/ و LDAP/SSO احراز هویت، Prometheus/Grafana نظارت، Kubernetes/Helm، و SQLCipher ذخیره‌سازی رمزنگاری شده با

چرا آن را ساختیم

ممکن است منقضی شود API چون بررسی صرفاً مبتنی بر تنظیمات قابل اعتماد نیست — کلید مدلی ممکن است منسوخ گردد، و یک فایل تنظیمات هیچ اطلاعاتی درباره تأخیر واقعی، خطاهای LLMsVerifier واقعی یا اینکه آیا مدل واقعاً ورودی شمارا می‌بیند و درک می‌کند، به شما نمی‌دهد جایگزین «در تنظیمات هست، پس حتماً کار می‌کند» با اثبات می‌شود: تنها مدل‌هایی که به‌طور قطعی پاسخ صحیح می‌دهند، به‌عنوان قابل استفاده علامت‌گذاری و صادر می‌شوند.

محتوا

چرا این یک تغییردهنده بازی است

می‌کند — واژه‌ای که به‌ندرت در فضایی پر از تنظیماتی قابل اعتماد را LLM این سامانه، ناوگان‌های که با سکوت دروغ می‌گویند، به دست می‌آید. به‌جای امید بستن به اینکه یک مدل پیکربندی‌شده کار کند، تیم‌ها تضمینی قابل آزمایش و اجباری دریافت می‌کنند که هر مدل در حال استفاده، آزمایش‌های واقعی را پشت سر گذاشته است؛ با نظارت، جایگزینی خودکار در صورت شکست و صافرات صرفاً این سامانه به تنها منبع حقیقت Helix، تأییدشده که حلقه‌ها از اثبات تا تولید می‌بندد. در اکوسیستم تبدیل می‌شود: سایر سرویس‌ها (از جمله LLM برای مدل‌ها، ارائه‌دهندگان و فراداده‌های تأیید بر اساس آن مسیریابی می‌کنند، به‌طوری که کل پلتفرم به‌جای آنکه هر تیم HelixTranslate) حس امپوزانه خود را داشته باشد، یک پاسخ صادقانه به این پرسش می‌دهد: «الان کدام مدل‌ها واقعاً کار می‌کنند؟»

نوآوری‌ها

- برای درک مطلب HTTP دروازه‌ای واقعی و مبتنی بر — «تأیید اجباری» آیا کد من را می‌بینی؟ که هر مدل باید پیش از قابل استفاده شدن از آن عبور کند؛ ویژگی متمایزکننده اصلی این محصول و دلیلی که هیچ چیز تأییدنشده‌ای از آن عبور نمی‌کند.
- تنها CLI و AI پیکربندی‌های تولیدشده برای ابزارهای — صافرات پیکربندی صرفاً تأییدشده شامل مدل‌هایی هستند که تأیید شده‌اند، بنابراین پیکربندی‌ای که ارسال می‌کنید نمی‌تواند به‌طور پنهانی یک مدل معیوب را دوباره وارد کند.
- هر ارائه‌دهنده تولیدشده/مدل، پسوندی قابل ردیابی دارد — (llmsvd) سیستم پسوند برندینگ که اصالت تأییدشده را در هر جایی که خروجی سفر می‌کند، نمایان می‌سازد.
- این سامانه به‌جای فرض — CLI در میان بسیاری از عامل‌ها و ارائه‌دهندگان تشخیص قابلیت‌ها (SSE، WebSocket، JSONL، EventStream) کردن، انواع جریان داده فشرده‌سازی و رفت‌وآمدهای کش را شناسایی می‌کند.

- قطع‌کننده‌های مدار، مسیریابی مبتنی بر تأخیر که هنگام عبور زمان تا — جایگزینی خودکار مقاوم دریافت اولین توکن از آستانه مشخصی، ترافیک را تغییر مسیر می‌دهد، پروب‌های سلامت و تقسیم ترافیک وزنی، ناوگان را حتی زمانی که ارائه‌دهندگان منفرد دچار نوسان می‌شوند، پاسخگو نگه می‌دارند.
- الگوی تجزیه ناظر/کارگر به همراه نقطه بازرسی و یکپارچگی حافظه — خودمختاری طولانی مدت جلسات طولانی‌ای را که در غیر این صورت بافتار را از بین می‌بردند، پایدار نگه می‌دارد.
- برای تقویت بافتار مبتنی بر داده‌های واقعی **RAG / vector-DB** یکپارچگی با

بزرگ‌ترین چالش‌های فنی و راه‌حل‌های ما

- هدف اصلی و اثبات اینکه یک مدل واقعاً کار می‌کند، نه فقط اینکه پیکربندی شده است برقرار می‌کند و API سخت‌ترین بخش کار. با آزمون اجباری دیدن کد حل شد که تماس‌های واقعی پاسخ‌ها را برای درک مطلب مثبت تحلیل می‌کند، پشتوانه‌اش مجموعه‌ای گسترده از آزمون‌های قابلیت است — و سپس با امتناع از صامرات هر چیزی که تأیید نشده باشد، اثبات (نه پیکربندی) را به دروزه تولید تبدیل می‌کند
- با یک هماهنگ‌کننده جایگزینی حل شد که **پایداری در برابر ارائه‌دهندگان شخص ثالث ناپایدار** **M** شکست در **N** ناپایداری ارائه‌دهندگان را حالت عادی فرض می‌کند: قطع‌کننده‌های مدار پس از ثانیه، یک ارائه‌دهنده را به عنوان معیوب علامت‌گذاری می‌کنند، مسیریابی مبتنی بر تأخیر از نقاط پایانی کند دوری می‌کند، پروب‌های سلامت دوره‌ای برای بازیابی جستجو می‌کنند و مسیریابی وزنی مدل‌های مقرون به‌صرفه را با مدل‌های ممتاز متعادل می‌سازد.
- با الگوی تجزیه ناظر/کارگر حل شد که کلرهای بزرگ را **پشتیبانی از جلسات طولانی و خودمختار** به قطعات قابل مدیریت تقسیم می‌کند، نقطه بازرسی دوره‌ای در ذخیره‌سازی ابری که پیشرفت را در **LLM +** برابر قطع شدن حفظ می‌کند و مدیریت لایه‌ای بافتار (بنجره لغزان + خلاصه‌سازی تا مدل بدون غرق شدن در توکن‌ها، موضوع را دنبال کند **RAG**)
- اختصاصی هر ارائه‌دهنده پشت **Go** با پنهان کردن بسیاری از آداپتورهای **پراکندگی ارائه‌دهندگان** — یک واسط مشترک حل شد، به‌طوری که نقاط پایانی واقعی به‌طور مرکزی فهرست می‌شوند. بنابراین افزودن یک ارائه‌دهنده تغییری محدود است، نه موجی در سراسر کدبیس.

محتوا

پشته فناوری

- به‌عنوان زبان اصلی پلتفرم انتخاب شده است؛ به دلیل قابلیت همزمانی، موتور تأیید — **Go**، چندرشته‌ای را به حرکت درمی‌آورد که می‌تواند به‌طور موازی مدل‌های متعددی را بررسی کند، به‌علاوه سرویس‌های پیرامونی.
- انتخاب شده است که احراز هویت، محدودسازی نرخ REST API به‌عنوان سرور — **Gin** را مدیریت می‌کند **WebSocket/SSE** درخواست‌ها و نقاط پایانی
- برای ذخیره‌سازی تعبیه‌شده با رمزنگاری سطح پایگاه داده — **SQLite + SQLCipher** انتخاب شده‌اند، زیرا داده‌های تأیید (کلیدها، نتایج) حساس هستند و باید به‌طور پیش‌فرض در حالت استراحت رمزنگاری شوند.
- به‌عنوان لایه کش انتخاب شده است تا جستجوهای داغ تأیید و فراداده‌ها با سرعت — **Redis** بالا انجام شوند.
- برای تأمین معماری رویدادمحور انتخاب شده‌اند: پیام‌رسانی و — **RabbitMQ + Kafka** جریان داده‌ای که تولیدکنندگان را از مصرف‌کنندگان در سراسر پلتفرم جدا می‌کند.
- برای ارتباطات قویاً تایپ‌شده بین سرویس‌ها و — **gRPC + Protocol Buffers** انتقال رویدادها بین مؤلفه‌ها انتخاب شده‌اند.

- **QUIC / HTTP-3 (quic-go)** — برای پشتیبانی از انتقال مدرن انتخاب شده‌اند — قابلیت‌های ارائه شده، نه HTTP/3 اسناد مخزن، در دسترس بودن (ادعای جهانی).
- **JWT + LDAP/NTLM** — برای احراز هویت سازمانی انتخاب شده‌اند تا پلتفرم در هویت ادغام شود (که در اسناد ادعا شده SSO/SAML/OIDC شرکتی موجود).
- **Viper (فشرده‌سازی)، Logrus (لاگینگ)، Brotli/compress (پیکربندی)** — زیرساخت عملیاتی: پیکربندی انعطاف‌پذیر، لاگ‌های ساختاریافته و فشرده‌سازی بار.
- **Angular** — برای برنامه تک‌صفحه‌ای وب انتخاب شده است؛ دروازه بصری برای تأیید و نظارت.
- **Python + SDKهای JavaScript** — مستند شده‌اند OpenAPI/Swagger انتخاب شده‌اند که از طریق.
- **Docker, Kubernetes, Helm** — برای استقرار تولید با نظارت سلامت و مقیاس‌بندی خودکار انتخاب شده‌اند تا ناوگان تأیید مانند هر سرویس مخرنی مقیاس‌پذیر باشد.
- **Prometheus + Grafana** — برای معیارها و داشبوردها انتخاب شده‌اند تا سلامت خود پلتفرم به اندازه مدل‌هایی که نظارت می‌کند قابل مشاهده باشد.
- **Testify (Go) + node -test/jsdom (وب)** — Go برای تست لایه‌ای در هسته — روابط کاربری وب انتخاب شده‌اند.

وضعیت و یادداشت‌های شفافیت

- واقعی را پیاده‌سازی می‌کند (یک سند قدیمی که تأیید HTTP تأیید Go کد منبع **وضعیت: بتا** (را صرفاً مبتنی بر پیکربندی توصیف می‌کند، آرمانی و منسوخ است — کد معتبر است).
- را اعلام کرده در حالی که یک برچسب MIT مجوز README فایل **مجوز: نامشخص** را ذکر کرده است — پیش از انتشار باید حل شود Apache-2.0 مجوز Dockerfile
- از «۱۲ آداپتور» سخن می‌گوید اما دایرکتوری راننده‌ندگان README تعداد راننده‌ندگان: فایل حدود ۲۶ مورد را فهرست کرده است — آن را «+۱۲ / در حال توسعه بیشتر» در نظر بگیرید معتبر go.mod فایل‌های وضعیت متعددی با برچسب «نهایی/کامل» وجود دارند؛ کد، مستندات و هستند.
- Helix قرار دارد، اما از نظر کارکردی لایه اعتماد زیرساخت vasic-digital مخزن در سازمان LLM است.

LLM/؛ منبع واحد حقیقت برای فراداده‌های LLM اصلی (خوشه زیرساخت-Helix: سطح اولویت قرار دارد HelixTrack راننده‌نده/تأیید). پس از